

■ GUÍA DE AGENTES XCIEN

Ecosistema de Inteligencia Artificial Operativa

Versión 1.0 · 15 May 2026 · Emitido por Antigravity

Este documento describe el ecosistema completo de agentes de IA y servicios operativos que integran la plataforma XCIEN 2026. Cubre arquitectura, roles, contexto de uso, comandos de activación y estado real verificado.

1. TIPOS DE AGENTES

Un agente en XCIEN es un programa Python con un rol empresarial específico. Existen tres categorías:

TIPO	DESCRIPCIÓN	EJEMPLO
■ Agente IA	Usa Claude (Anthropic) para razonar y generar respuestas	director_general_v2.py
■ ■ Agente Operativo	Lógica de negocio con datos reales, sin IA	wfm_workflow_service.py
■ Agente Stub	Estructura lista, pendiente de implementación completa	agent_noc.py

2. ARQUITECTURA DEL ECOSISTEMA

JERARQUÍA DE CONTROL

CAPA	AGENTE / ROL	TECNOLOGÍA	CANAL
Humano	Miguel Macías — Director General	—	Conversación
Orquestador	Antigravity — Ingeniero	Gemini/Claude	IDE + Terminal
Portal Web	Director General V2	claude-sonnet-4-6	localhost:8000
CLI Orquestador	Simulador / Director General	claude-haiku-4-5	Terminal
Sub-agentes (×8)	NOC, Ops, RRHH, Odoo, Dispatch, Comercial, Preventa, Legal	claude-haiku-4-5	CLI (delegado)
Servicios	WFM, Assets, Tokens, Odoo	Python + JSON/xmlrpc	API REST :8000

FLUJO DE COMUNICACIÓN

```
Miguel → Antigravity → Portal (:8000) / CLI
Portal → Director General V2 → API REST → Servicios (WFM / Assets / Tokens / Odoo)
CLI → Simulador → delega por palabras clave → 8 Agentes Especializados → Servicios
```

3. GUÍA DE USO — AGENTES IA

■ Director General V2

Archivo	director_general_v2.py
Modelo	claude-sonnet-4-6
Contexto	Portal Web · localhost:8000

Función: Cerebro ejecutivo del portal. Responde con datos reales de WFM (tickets, técnicos), el Libro Maestro de Operaciones y el estado de la red. Recomienda asignaciones, analiza urgencias y genera reportes ejecutivos.

Cuándo usarlo: Cuando necesitas información de operaciones, estado de tickets, análisis de técnicos disponibles o cualquier consulta de alto nivel.

```
# Arrancar el portal completo:
python3 backend/servidor_academia.py

# Consulta vía API:
curl -X POST http://localhost:8000/api/director/chat \
-H 'Content-Type: application/json' \
-d '{"message": "dame el estado de los tickets"}'
```

■ Simulador — Director General CLI

Archivo	simulador_agentes.py
Modelo	claude-haiku-4-5-20251001
Contexto	CLI · Terminal

Función: Orquestador de los 8 agentes especializados. Recibe una instrucción en lenguaje natural y la delega automáticamente al agente correcto según palabras clave detectadas.

Cuándo usarlo: Cuando necesitas que un agente especializado (NOC, Legal, Comercial, etc.) analice algo desde la terminal.

```
cd /Users/mesquite/Antigravity/backend/agents
python3 simulador_agentes.py
```

■ DevOps Agent

Archivo	devops_agent.py
Modelo	claude-sonnet-4-6
Contexto	CLI · Diagnóstico del sistema

Función: Revisa el estado real del sistema: Git, PM2, Odoo, métricas del servidor. Analiza los resultados con Claude y genera un diagnóstico ejecutivo.

Cuándo usarlo: Cuando algo falla en producción o necesitas un health-check completo del sistema.

```
cd /Users/mesquite/Antigravity/backend/agents
python3 devops_agent.py
```

■ Creador de Manuales

Archivo	creador_manuales.py
Modelo	claude-sonnet-4-6
Contexto	CLI · Generación de documentos

Función: Genera manuales técnicos en formato Markdown listos para la Biblioteca del portal. Usa el contexto de XCIEN para producir documentación profesional.

Cuándo usarlo: Cuando necesitas crear o actualizar manuales técnicos para el portal.

```
cd /Users/mesquite/Antigravity/backend/agents
python3 creador_manuales.py
```

■ Auditor de Código

Archivo	auditor_codigo.py
Modelo	claude-sonnet-4-6
Contexto	CLI · Revisión de calidad

Función: Audita archivos de código buscando problemas de higiene, seguridad y arquitectura. Emite un veredicto: PASA o REQUIERE REFACTOR.

Cuándo usarlo: Antes de un despliegue o cuando sospechas que un archivo tiene problemas.

```
# Auditar archivo específico:
python3 auditor_codigo.py --file backend/agents/director_general_v2.py

# Listar archivos auditables:
python3 auditor_codigo.py --list

# Auditoría global:
python3 auditor_codigo.py --full
```

4. ROUTING DEL SIMULADOR — 8 AGENTES

El Simulador detecta palabras clave en tu mensaje y delega automáticamente al agente correcto:

PALABRAS CLAVE (en tu mensaje)	AGENTE ASIGNADO	ROL
caído, enlace, noc, red, falla, monitoreo, nodo	■ Agente NOC	Network Operations Center
campo, EPP, instalación, técnico, seguridad, mástil	■ Agente Operaciones	Field Service Management
capacitación, curso, academia, examen, manual	■ Agente RRHH/Academia	Capacitación y Certificación
odoo, erp, módulo, ticket, registro	■ Agente Odoo	ERP wispi17
asignar, ruta, despacho, agenda, dispatch	■ Agente Dispatch	Logística de campo
venta, comercial, cotización, prospecto, cliente	■ Agente Comercial	Ventas y CRM
factibilidad, preventa, coordenadas, antena, señal	■ Agente Preventa	Factibilidad técnica
contrato, legal, IFT, arrendamiento, SLA, cláusula	■ ■ Agente Legal	Asesoría legal telecom

5. SERVICIOS OPERATIVOS (SIN IA)

SERVICIO	FUNCIÓN	ENDPOINTS PRINCIPALES
WFMWorkflowService	Motor de órdenes de trabajo (8 etapas: Solicitud → Cierre)	/api/wfm/tickets /api/wfm/tecnicos /api/wfm/asignar/{id}
AssetService	Control de activos físicos (antenas, equipos, sites)	/api/activos/registrar /api/activos/listar /api/activos/{id}/label
TokenService	X-Token economy XCIEN (oportunidades, RRHH)	/api/tokens/emitir /api/tokens/operativo /api/tokens/listar
TransaccionesService	Flujos intercompany entre empresas del grupo	/api/transacciones/registrar /api/transacciones/resumen
OdooConnector	Conexión XML-RPC a base wispi17	Usado internamente por otros servicios
LabelService	Etiquetas QR/PDF para activos y tickets	/api/activos/{id}/label /api/transacciones/{id}/label

6. FLUJO COMPLETO DE UNA ORDEN WFM

Cada instalación de cliente pasa por 8 etapas gestionadas por WFMWorkflowService:

#	ETAPA	RESPONSABLE	DESCRIPCIÓN
1	Solicitud Comercial	Área Comercial	El cliente pide el servicio. Se genera la orden.
2	Preventa	Agente Preventa	Análisis de factibilidad técnica: coordenadas, señal, cobertura.
3	Contratación	Legal + Dirección	Firma del contrato. Aprobación formal de la orden.
4	Almacén	Logística	Solicitud de equipos: router, cables, antena, accesorios.
5	Aprovisionamiento	NOC / TI	Configuración de red: IP, VLAN, BW, gateway, firmware.
6	Instalación	Técnico de Campo	Trabajo en sitio con checklist, evidencias fotográficas y prueba de velocidad.
7	Alta NOC	NOC	Alta del cliente en el sistema de monitoreo. Ping confirmado.
8	Cierre	Técnico + Coordinador	Entrega formal. Firma del cliente. Ticket cerrado.

7. ESTADO REAL — VERIFICADO 2026-05-15

AGENTE	ESTADO	MODELO	CONTEXTO
director_general_v2	■ OPERATIVO	claude-sonnet-4-6	Portal :8000
simulador_agentes (×8)	■ OPERATIVO	claude-haiku-4-5	CLI
devops_agent	■ LISTO	claude-sonnet-4-6	CLI
creador_manuales	■ LISTO	claude-sonnet-4-6	CLI
auditor_codigo	■ LISTO	claude-sonnet-4-6	CLI
wfm_workflow_service	■ OPERATIVO	— (Odo+JSON)	API :8000
asset_service	■ OPERATIVO	—	API :8000
token_service	■ OPERATIVO	—	API :8000
transacciones_service	■ OPERATIVO	—	API :8000
odoo_connector	■ OPERATIVO	— (xmlrpc)	Servicio interno
puente (bridge)	■■ CONDICIONAL	—	Requiere :8000 activo
telegram_bot	■ SIN TOKEN	—	Pendiente configurar
agent_noc / wfm / academia...	■■ STUBS	—	Pendiente implementar

8. COMANDOS RÁPIDOS — REFERENCIA

Arrancar el portal completo (backend + portal)

```
cd /Users/mesquite/Antigravity
lsof -ti :8000 | xargs kill -9 2>/dev/null
python3 backend/servidor_academia.py
# → http://localhost:8000
```

Correr el Simulador de 8 Agentes

```
cd /Users/mesquite/Antigravity/backend/agents
python3 simulador_agentes.py
```

Auditar un archivo de código

```
python3 auditor_codigo.py --file backend/agents/director_general_v2.py
```

Ver estado del puente (bridge)

```
python3 puente.py --query
```

Actualizar tarea en el dashboard

```
python3 puente.py --task 'Procesando facturas' --status working
```

9. REGLAS DE ORO DEL ECOSISTEMA

#	REGLA	POR QUÉ
1	No duplicar dashboards — toda nueva funcionalidad va en Xcien2	Evitar fragmentación y mantener una sola fuente de verdad visual
2	Usar variables CSS: var(--xcien-accent), var(--xcien-bg)...	El sistema de temas cambia colores globalmente; hex hardcodeado lo rompe
3	El Hub Principal (/) es sagrado — no se elimina	Es la puerta de entrada del sistema para todos los usuarios
4	ESTADO_SISTEMA.md es la fuente de verdad — actualizar en cada cambio	Garantiza que cualquier IA que tome el contexto tenga información correcta
5	Antigravity escribe código, no toma decisiones operativas	Las decisiones operativas son del Director General humano (Miguel)
6	Verificar el servidor antes de hacer cambios	El sistema puede estar en producción; un cambio sin verificar puede romperlo